

UNIVERSIDAD SAN CARLOS DE GUATEMALA

FACULTAD DE CIENCIAS DE LA INGENIERÍA

INTELIGENCIA ARTIFICIAL 1

ING. LUIS FERNANDO ESPINO

AUX. JOSE ANDRES MONTENEGRO SANTOS



DOCUMENTACIÓN FASE 3

POR:

201230463 - 2253 62503 0901	-SUM COYOY, RANDY MARCELINO
202031064 - 3133 35931 0901	-OVALLE DE LEÓN, CARLOS ALEXIS
202031683 - 3353 15267 0901	-GORDILLO GONZÁLEZ , PEDRO RICARDO

GUATEMALA, GUATEMALA, DICIEMBRE DE 2024

Índice

Descripción del Proyecto: Chatbot con Tensor Flow.....	3
Concepto del Chatbot.....	3
Versiones Utilizadas.....	3
Requisitos previos.....	3
Configuración.....	4
Cargar y Preprocesar los datos.....	4
Carga de datos.....	4
Normalización.....	5
Preprocesamiento de datos.....	5
Codificación del vocabulario.....	6
Codificación de texto.....	6
Construcción del modelo.....	6
Arquitectura.....	7
Entrenamiento del modelo.....	7
Traducción de las entradas.....	7
Estructura del Proyecto Interfaz Gráfica.....	8
Estructura del código.....	8
Configuración inicial.....	8
Componentes de la GUI.....	8
Frames y Widgets.....	9
Funciones Clave.....	9
Agregar nuevo Chat.....	9
Cerrar Chat.....	9
Enviar Mensaje.....	10
Guardar Chat.....	10
Resaltar Texto.....	11
Organizamiento de Layout.....	11
Versionamiento semántico.....	11

Descripción del Proyecto: Chatbot con Tensor Flow

Descargar el proyecto del repositorio:

git clone [git@github.com:Peter2510/IA1_Proyecto_10.git](https://github.com/Peter2510/IA1_Proyecto_10.git)

Este proyecto consiste en la creación de un chatbot que utiliza un modelo de aprendizaje profundo desarrollado en TensorFlow, junto con una interfaz gráfica implementada en Tkinter. Su objetivo principal es responder automáticamente a las consultas del usuario en función de un conjunto de datos de entrenamiento predefinido.

Concepto del Chatbot

El chatbot utiliza una arquitectura secuencial basada en una red neuronal con capas densas. Se entrena con datos pre procesados para generar respuestas.

Para poder utilizar y ejecutar el proyecto debe de dirigirse a la carpeta backend 3.0 y seguir los siguientes pasos.

Versiones Utilizadas

- Python 3.7 -.11
- Tensor Flow 2.12
- NLTK
- Tkinter(Integrado)

Requisitos previos

- Python
- Tensor Flow
- Numpy
- NLTK
- Tkinter

Configuración

Crea un entorno virtual con el comando

```
python3 -m venv venv
```

Activa el entorno

```
source venv/bin/activate
```

instala TensorFlow

```
pip install tensorflow==2.12
```

Ejecuta con el comando:

```
python3 botsi.py
```

Cargar y Preprocesar los datos

El chatbot utiliza un archivo JSON llamado original-dialogues.json que se encuentra en la estructura del proyecto

Carga de datos

El archivo JSON se carga en Python usando el módulo json

```
Peter2510, 1 hour ago | 1 author (Peter2510)
import tensorflow as tf      Peter2510, 2 days ago • Agregando backend
import json
import os
import numpy as np

# Cargar los datos originales
with open("original-dialogues.json", "r", encoding="utf8") as file:
    original_data = json.load(file)
```

Normalización

La función `normalize_text` convierte las frases a minúsculas, elimina acentos y asegura una compatibilidad UTF-8

```
# Función para normalizar solo las propiedades de texto
def normalize_text(data):
    for item in data:
        if isinstance(item.get("text"), str):
            item["text"] = (
                item["text"]
                .encode("utf-8")
                .decode("utf-8")
                .lower()
                .replace("á", "a")
                .replace("é", "e")
                .replace("í", "i")
                .replace("ó", "o")
                .replace("ú", "u")
            )
    return data
```

Preprocesamiento de datos

La función `preprocess_data` toma un conjunto de frases normalizadas y genera pares consecutivos

```
# Preprocesar los datos para convertir diálogos en pares de entrenamiento
def preprocess_data(data):
    pairs = []
    for i in range(len(data) - 1):
        pairs.append({"input": data[i]["text"], "output": data[i + 1]["text"]})
    return pairs

pairs = preprocess_data(data)
```

Codificación del vocabulario

la función `create_vocabulary` crea un conjunto único de palabras presentes en las entradas

```
# Crear vocabulario
def create_vocabulary(data):
    vocab = set()
    for text in data:
        for word in text.split():
            vocab.add(word.lower())
    return list(vocab)

vocab = create_vocabulary([pair["input"] for pair in pairs])
vocab_size = len(vocab)
```

Codificación de texto

la función `encode_text` convierte un texto en un vector binario utilizado en el vocabulario

```
# Codificar texto a vectores
def encode_text(text, vocab):
    encoded = np.zeros(len(vocab), dtype=np.float32)
    for word in text.split():
        if word in vocab:
            index = vocab.index(word)
            encoded[index] = 1
    return encoded
```

Construcción del modelo

El modelo es una red neuronal secuencial con dos capas densas:

```
# Crear el modelo
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(128, activation="relu", input_shape=(vocab_size,)),
        tf.keras.layers.Dense(len(unique_responses), activation="softmax")
    ])
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model

model = create_model()
```

Arquitectura

Capa Densa(128 neuronas): Activa las características más relevantes

Capa de salida(Softmax): Genera probabilidades para cada respuesta posible

Entrenamiento del modelo

Epoch: XX iteraciones completas sobre los datos

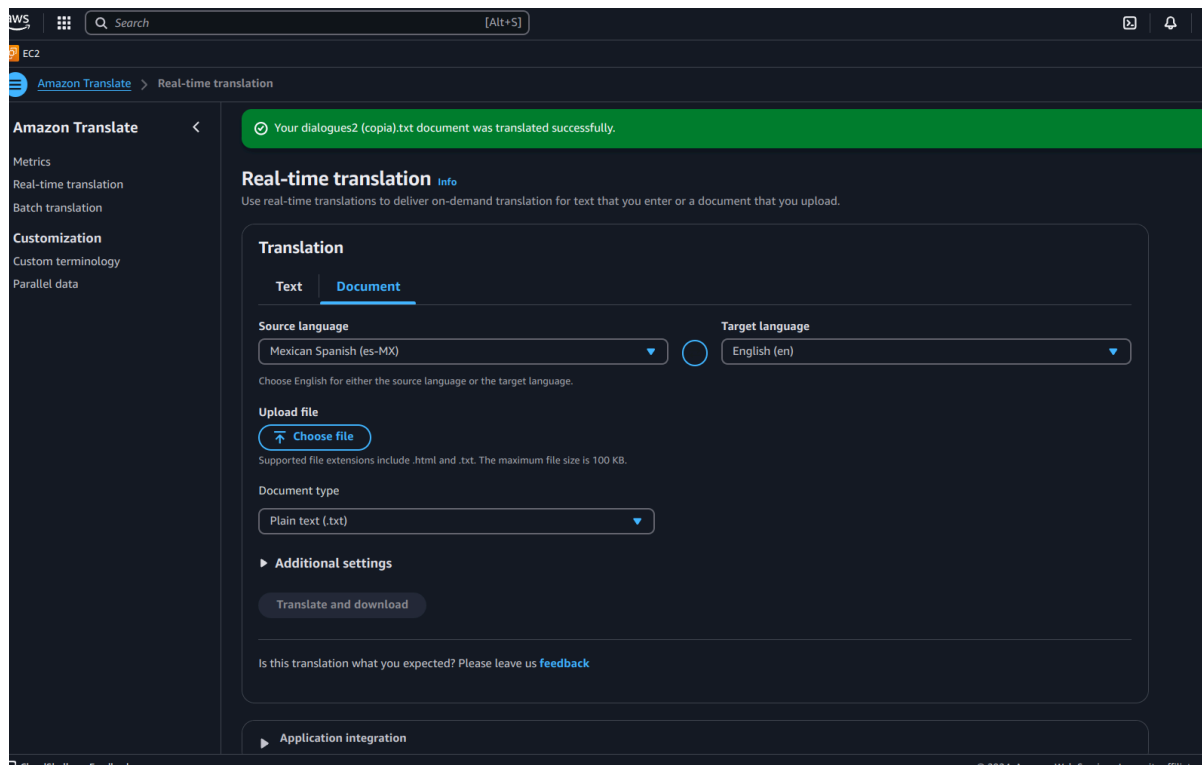
Batch Size: XX muestras por lote

Función de pérdida: Categorical Cross Entropy

Optimizador: Adam

Traducción de las entradas

Para realizar la traducción de las entradas, se hizo uso del servicio de AWS Amazon Translate, éste obtiene como entrada un archivo en español y devuelve como salida un archivo en su equivalente en inglés, esto para que sea una traducción fiable y confiable, ya que es un servicio especializado de traducción de texto.



EC2

Amazon Translate > Batch translation

Amazon Translate

Metrics

Real-time translation

Batch translation

Customization

Custom terminology

Parallel data

traduccion-ia translation job is in progress

Large batches may take a while to finish. Check the status again soon.

Translation jobs (1) Info

Search

Name	Source language	Target Languages	Started
traduccion-ia	Mexican Spanish (es-MX)	en	January 3, 2025, 01:10 (UTC-06:00)

aws

Search [Alt+S]

EC2

Amazon Translate > Batch translation > traduccion-ia

Amazon Translate

Metrics

Real-time translation

Batch translation

Customization

Custom terminology

Parallel data

traduccion-ia translation job is complete

You can find your output files in Output file location.

traduccion-ia

Job details

Status

Completed

Started

January 3, 2025, 01:10 (UTC-06:00)

Ended

January 3, 2025, 01:28 (UTC-06:00)

Source language

Mexican Spanish (es-MX)

Target language

English (en)

File format

text/plain

Estructura del Proyecto Interfaz Gráfica

La GUI permite la interacción con botsi en múltiples sesiones de chat.

Ofrece las siguientes funcionalidades:

- Crear nuevas pestañas para sesiones de chat.
- Enviar mensajes al bot y recibir respuestas.
- Guardar el historial de chat como archivos .txt.
- Resaltar texto seleccionado dentro del chat.
- Personalización visual mediante colores y fuentes.

Estructura del código

Configuración inicial

Ventana principal: Configurada con root, contiene propiedades como tamaño (800x700), posición centrada en pantalla, y restricciones de redimensionamiento.

Colores: Variables para gestionar temas (fondo oscuro, texto claro).

```
import tkinter as tk
from tkinter import ttk, filedialog
from datetime import datetime

# Configuración principal
root = tk.Tk()
root.title("Botsi")
root.geometry("800x700")
root.resizable(False, False)

# Colores
BG_COLOR = "#222831"
TEXT_COLOR = "#eeeeee"
ENTRY_BG = "#393e46"
BUTTON_BG = "#00adb5"
BUTTON_FG = "#ffffff"
CHAT_BG = "#30475e"
CHAT_ENTRY_BG = "#1c1e21"
USER_BG = "#00adb5" # Fondo para los mensajes del usuario
BOT_BG = "#1c1e21"  # Fondo para los mensajes del bot
```

Componentes de la GUI

Usamos ttk.Notebook para manejar múltiples sesiones de chat dentro de una misma ventana.

```
# Notebook para múltiples chats
notebook = ttk.Notebook(root)
notebook.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
```

Frames y Widgets

Cada pestaña tiene un frame con:

- Área de texto (tk.Text) para mostrar el historial del chat.
- Barra de desplazamiento (tk.Scrollbar).
- Botón para cerrar la pestaña.

Funciones Clave

Agregar nuevo Chat

La función crea una nueva pestaña con:

Un cuadro de texto para el chat.

Un botón para cerrar la pestaña.

Configuración de etiquetas para resaltar mensajes del usuario, del bot y selecciones.

```
def agregar_chat():
    chat_id = f"Chat {len(chats) + 1}"
    frame = tk.Frame(notebook, bg=CHAT_BG)

    # Crear el botón de cerrar
    close_button = tk.Button(frame, text="X", bg="#FF5733", fg="ffffff", font=("Helvetica", 10, "bold"), bd=0, command=lambda: cerrar_chat(chat_id))
    close_button.pack(side=tk.TOP, anchor="ne", padx=5, pady=5)

    notebook.add(frame, text=chat_id)

    # Scrollbar
    scrollbar = ttk.Scrollbar(frame, orient=tk.VERTICAL)
    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

    # Cuadro de texto para el chat
    chat_text = tk.Text(
        frame,
        bg=CHAT_BG,
        fg=TEXT_COLOR,
        font=("Helvetica", 12),
        wrap=tk.WORD,
        yscrollcommand=scrollbar.set,
        bd=0,
        padx=10,
        pady=10,
        state="disabled", # Iniciar en modo de solo lectura
    )
    chat_text.pack(fill=tk.BOTH, expand=True)
    scrollbar.config(command=chat_text.yview)

    # Configurar etiquetas para el fondo de los mensajes
    chat_text.tag_configure("user", background=USER_BG, foreground=TEXT_COLOR, justify=tk.LEFT)
    chat_text.tag_configure("bot", background=BOT_BG, foreground=TEXT_COLOR, justify=tk.RIGHT)
    chat_text.tag_configure("highlight", background="#FFCCCB") # Resaltado en rojo claro

    # Vincular el evento para resaltar la selección
    chat_text.bind("<B1-Motion>", resaltar_seleccion)

    # Guardar el texto de chat en el diccionario
    chats[chat_id] = chat_text
```

Cerrar Chat

Elimina la pestaña y su contenido asociado.

```
# Función para cerrar un chat
def cerrar_chat(chat_id):
    notebook.forget(chats[chat_id].master)
    del chats[chat_id]
```

Enviar Mensaje

Obtiene el texto ingresado por el usuario.

Muestra el mensaje en el área de texto con un formato específico.

Invoca al modelo de IA para generar una respuesta y la agrega al chat.

```
# Función para enviar mensajes
def enviar_mensaje(event=None):
    user_msg = entry.get().strip()
    if user_msg:
        current_tab = notebook.tab(notebook.select(), "text")
        chat_text = chats[current_tab]

        # Habilitar temporalmente el cuadro de texto para agregar contenido
        chat_text.config(state="normal")

        # Mensaje del usuario
        timestamp = datetime.now().strftime("[%d-%m-%Y %H:%M:%S]")
        chat_text.insert(tk.END, f"Tú {timestamp}:\n{user_msg}\n\n", "user")

        # respuesta del chatbot (derecha)
        encoded_input = encode_text(user_msg, vocab)
        prediction = model.predict(np.array([encoded_input]))
        response_index = np.argmax(prediction)
        print(index_to_response.get(response_index, "No entiendo eso. "))
        bot_msg = index_to_response.get(response_index, "No entiendo eso.")
        chat_text.insert(tk.END, f"Botsi {timestamp}:\n{bot_msg}\n\n", "bot")
        chat_text.tag_config("bot", justify="right", foreground=TEXT_COLOR, font=("Helvetica", 12))

        # Deshabilitar nuevamente el cuadro de texto
        chat_text.config(state="disabled")

        # Asegurar que el texto sea visible
        chat_text.yview(tk.END)
        entry.delete(0, tk.END)
```

Guardar Chat

Permite guardar el contenido del chat actual en un archivo de texto utilizando `filedialog.asksaveasfilename`.

```
# Función para guardar el chat actual
def guardar_chat():
    current_tab = notebook.tab(notebook.select(), "text")
    chat_text = chats[current_tab]

    chat_content = chat_text.get("1.0", tk.END).strip()

    if chat_content:
        file_path = filedialog.asksaveasfilename(
            defaultextension=".txt",
            filetypes=[("Archivos de texto", "*.txt")],
            title="Guardar Chat",
            initialfile=current_tab,
        )
        if file_path:
            with open(file_path, "w", encoding="utf-8") as file:
                file.write(chat_content)
```

Resaltar Texto

Agrega una etiqueta de resaltado para el texto seleccionado en el chat.

```
# Función para resaltar texto seleccionado
def resaltar_seleccion(event):
    current_tab = notebook.tab(notebook.select(), "text")
    chat_text = chats[current_tab]

    try:
        # Obtener el rango seleccionado
        start = chat_text.index(tk.SEL_FIRST)
        end = chat_text.index(tk.SEL_LAST)

        # Eliminar cualquier resaltado anterior
        chat_text.tag_remove("highlight", "1.0", tk.END)

        # Aplicar la etiqueta de resaltado
        chat_text.tag_add("highlight", start, end)
    except tk.TclError:
        # Si no hay selección, eliminar el resaltado
        chat_text.tag_remove("highlight", "1.0", tk.END)
```

Organizamiento de Layout

Frame Izquierdo: Contiene el botón para agregar nuevos chats.

Frame Inferior: Maneja la entrada del usuario (Entry) y botones de acción (Enviar, Guardar Chat).

Integración con el modelo de IA

Botsi utiliza un modelo de aprendizaje automático que:

- Codifica el texto del usuario.
- Predice una respuesta usando un modelo pre entrenado.
- Mapea el índice de predicción a una respuesta del bot.

```
#respuesta del chatbot (derecha)
encoded_input = encode_text(user_msg, vocab)
prediction = model.predict(np.array([encoded_input]))
response_index = np.argmax(prediction)
print(index_to_response.get(response_index, "No entiendo eso. "))
bot_msg = index_to_response.get(response_index, "No entiendo eso. ")
chat_text.insert(tk.END, f"Botsi {timestamp}:\n{bot_msg}\n\n", "bot")
chat_text.tag_config("bot", justify="right", foreground=TEXT_COLOR, font=("Helvetica", 12))
```

Agregando backend 3.0

db2b929 < ? ? ? ? ? ?

+ bot_msg = index

Changes added in db2b929

Peter2510, 2 days ago

Versionamiento semántico

Python Version 3.10

Tensorflow 2.12

Numpy 1.26.4